

BREAKOUT DISCUSSION

Zero-Knowledge Proofs for Verifying Safety Properties of AI Models

What verifying a model would take, and where it breaks.

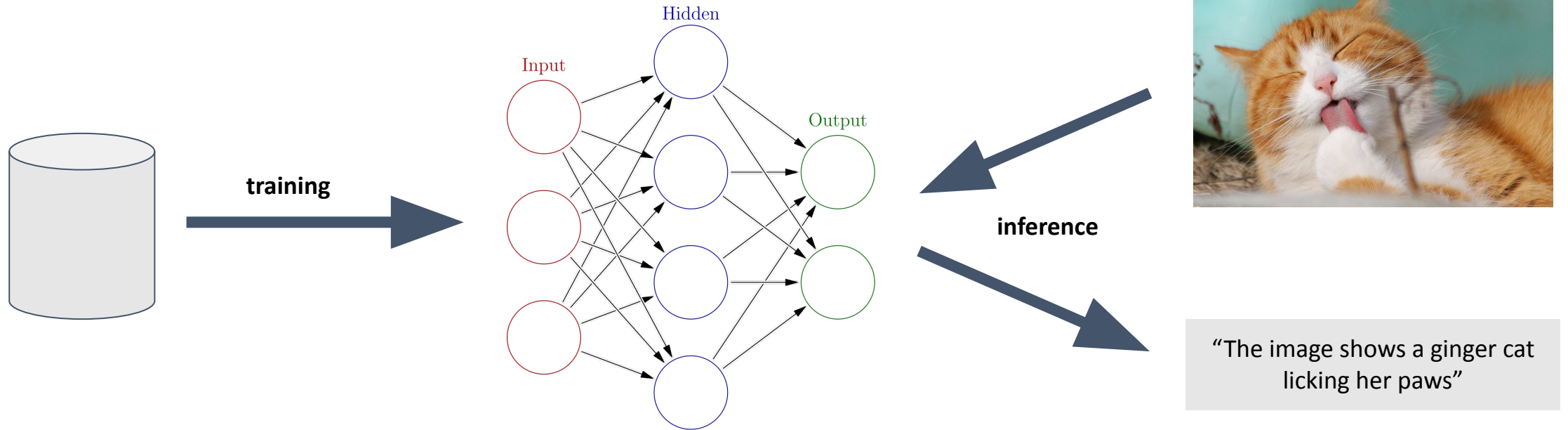
Pascal Berrang

University of Birmingham · Zeroth Research

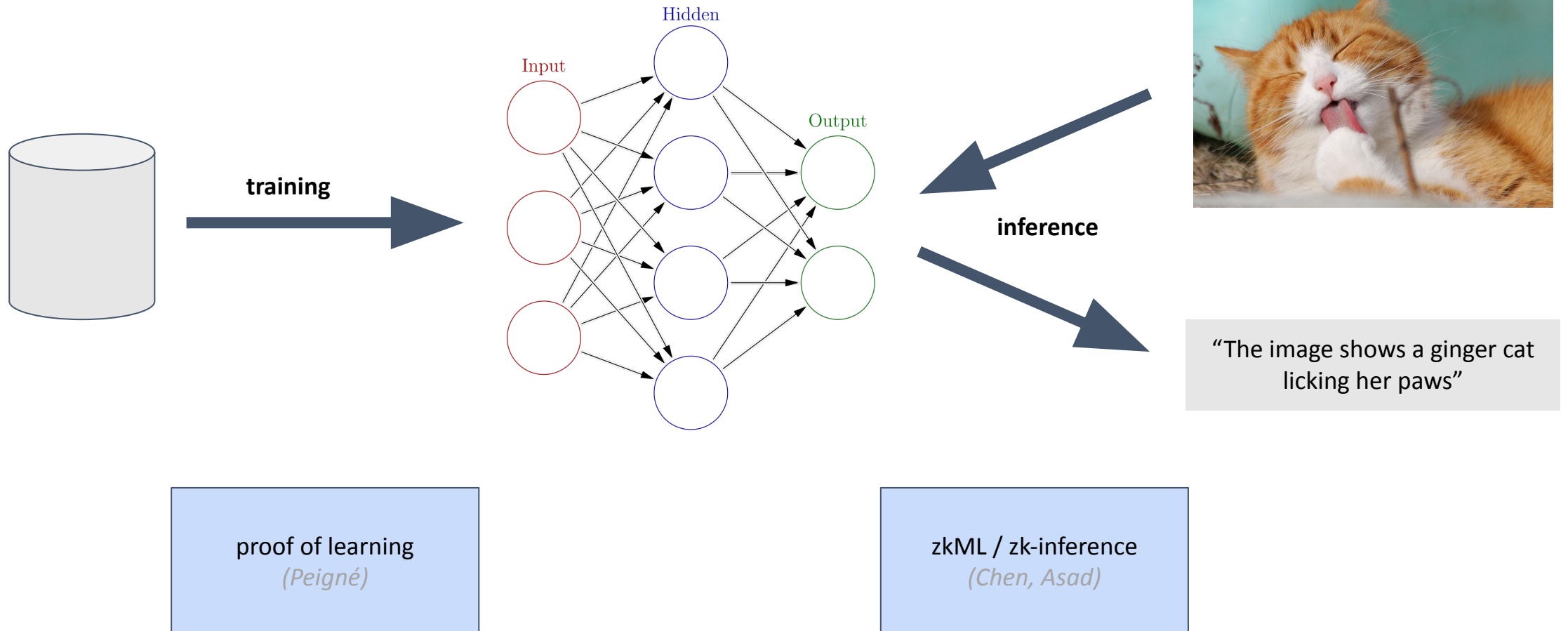
AViD Workshop · San Francisco · 17 May 2026

joint work with Mirco Giacobbe, Jacob Swales, Xiao Yang

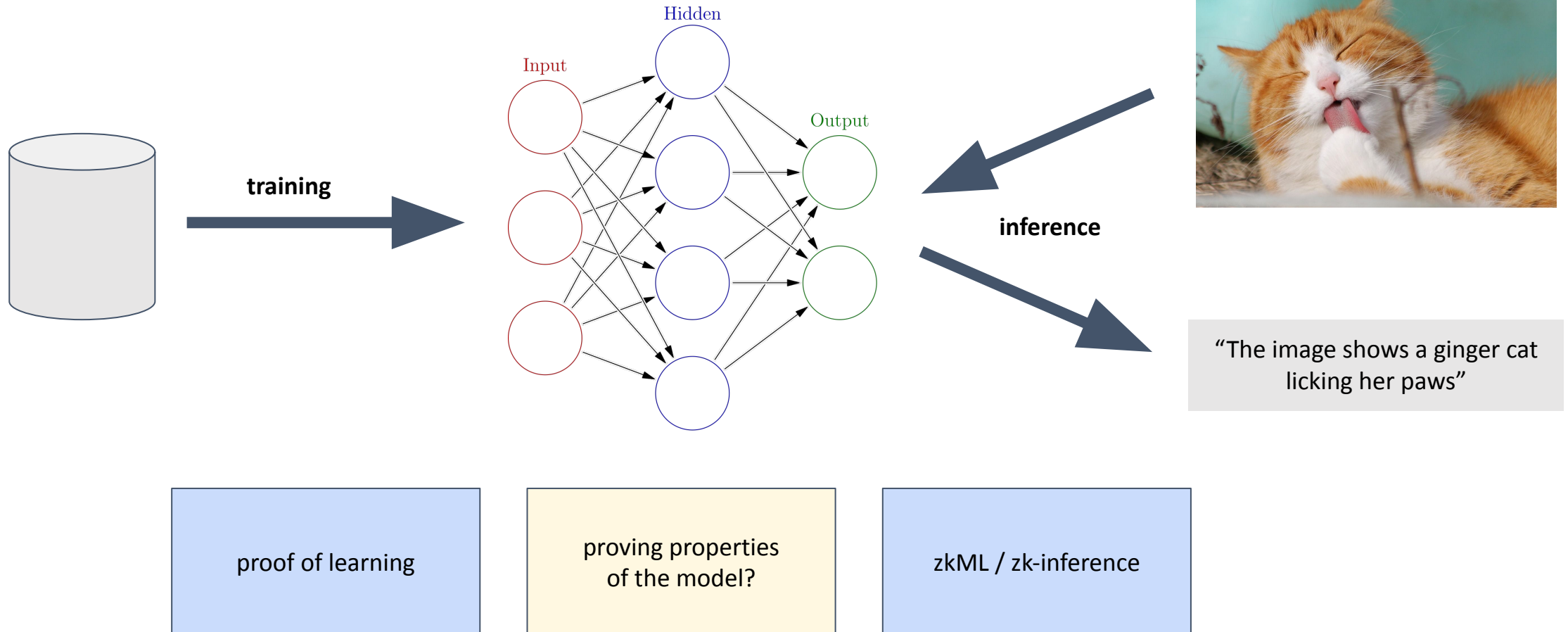
Most verification of AI is about processes.



Most verification of AI is about processes.



Most verification of AI is about processes.



Can we prove an (AI) system satisfies a formal spec?

REGULATED SOFTWARE

Avionics, medical devices

Vendor must prove compliance with a formal standard to a regulator.

PROTOCOL INTEROP

Two parties, no shared internals

Each side proves their implementation meets a joint spec.

AI SAFETY

Behavioural safety on a model

Model provider proves a deployed model satisfies a safety property.

There's a lot of ongoing work to adapt classical formal verification to AI systems.

Can we prove an (AI) system satisfies a formal spec *without revealing the system?*



Classical formal verification:
the verifier sees the system.

AI SAFETY ← this talk

Behavioural safety on a model

Model provider proves a deployed model satisfies a safety property — **without revealing weights, architecture, or training data.**

Ingredients: Certificate + Commitment + ZK Proof

Prove $M \models \phi$ without revealing M .

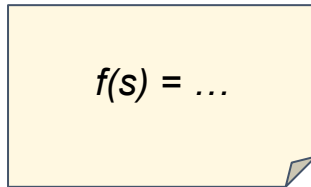
1

model-checking layer

Certification

Proof certificate v that M satisfies ϕ .

→ *ranking function*

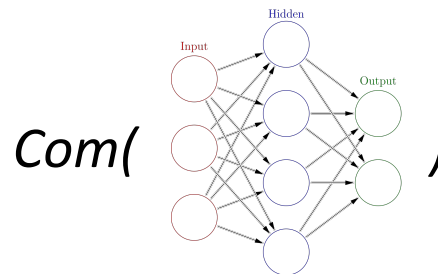


2

cryptographic binding

Commitment

Bind to M with a hiding + binding *commitment*. M becomes a fixed target the prover can no longer swap out, but the verifier never sees.



3

hiding the system

Zero-knowledge proof

Prove certificate over commitment.

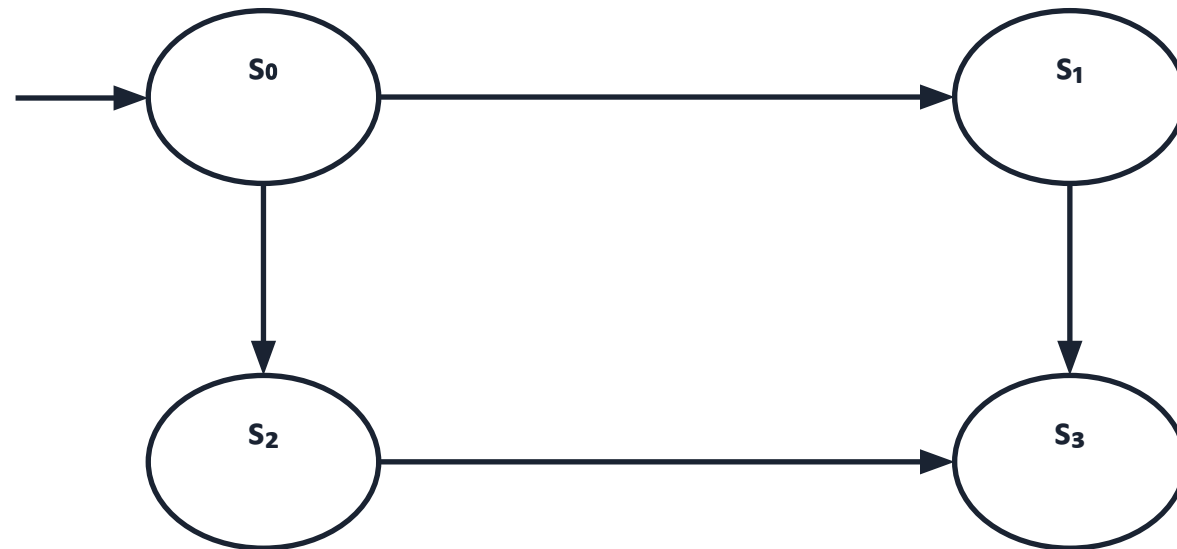
$M \models \phi$ where $C_M = Com(M)$

public: ϕ, C_M *cert*
private: M

Ranking Functions

For the special case of termination (toy example).

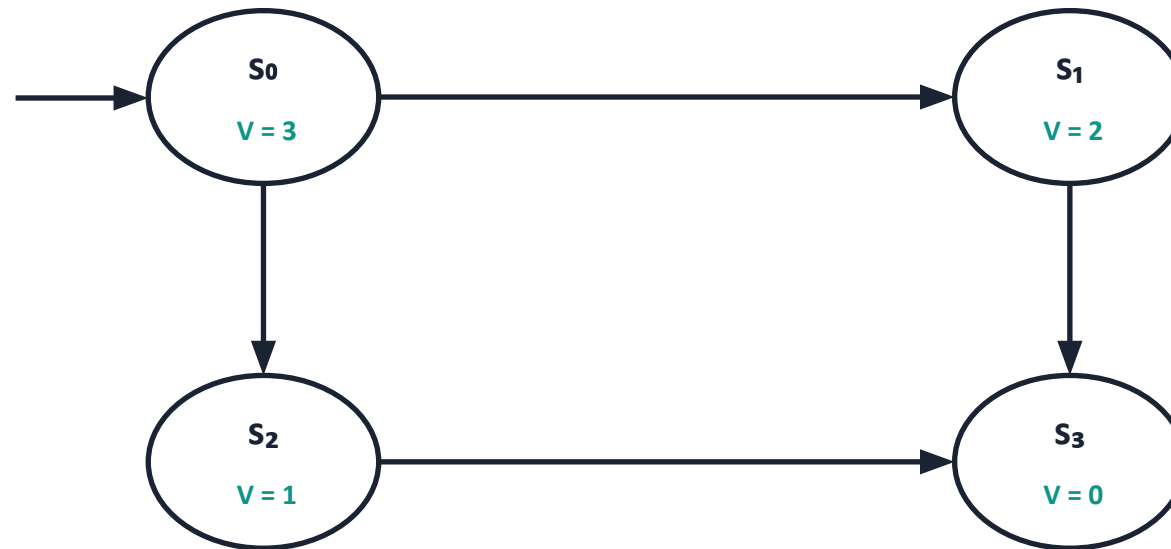
Show that this system terminates.



Ranking Functions

For the special case of termination (toy example).

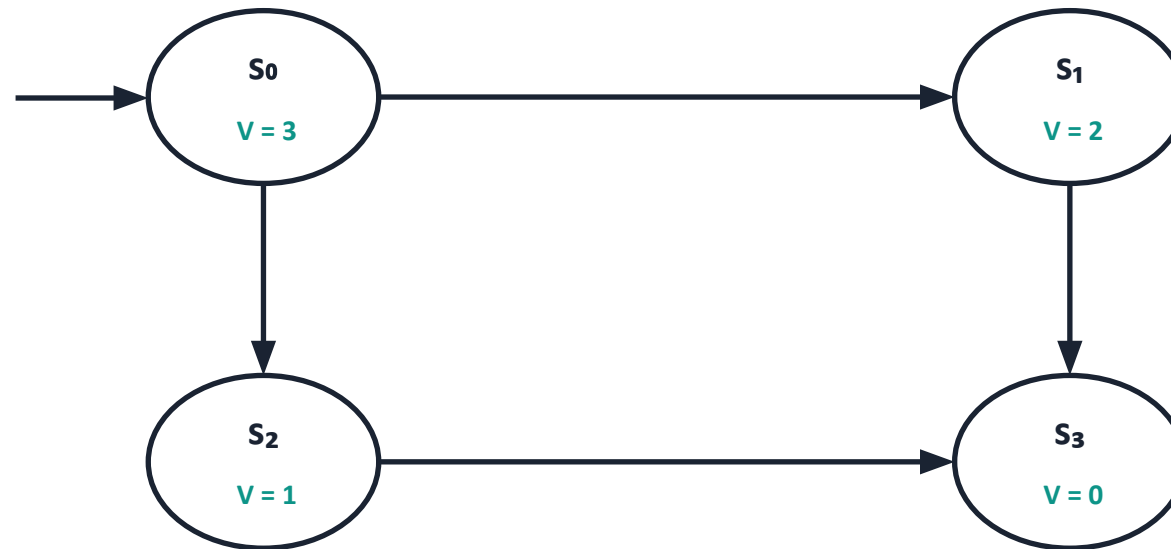
Show that this system terminates.



Ranking Functions

For the special case of termination (toy example).

Show that this system terminates.



(i) Initial states have finite V

no run starts at ∞

(ii) V strictly decreases on every transition

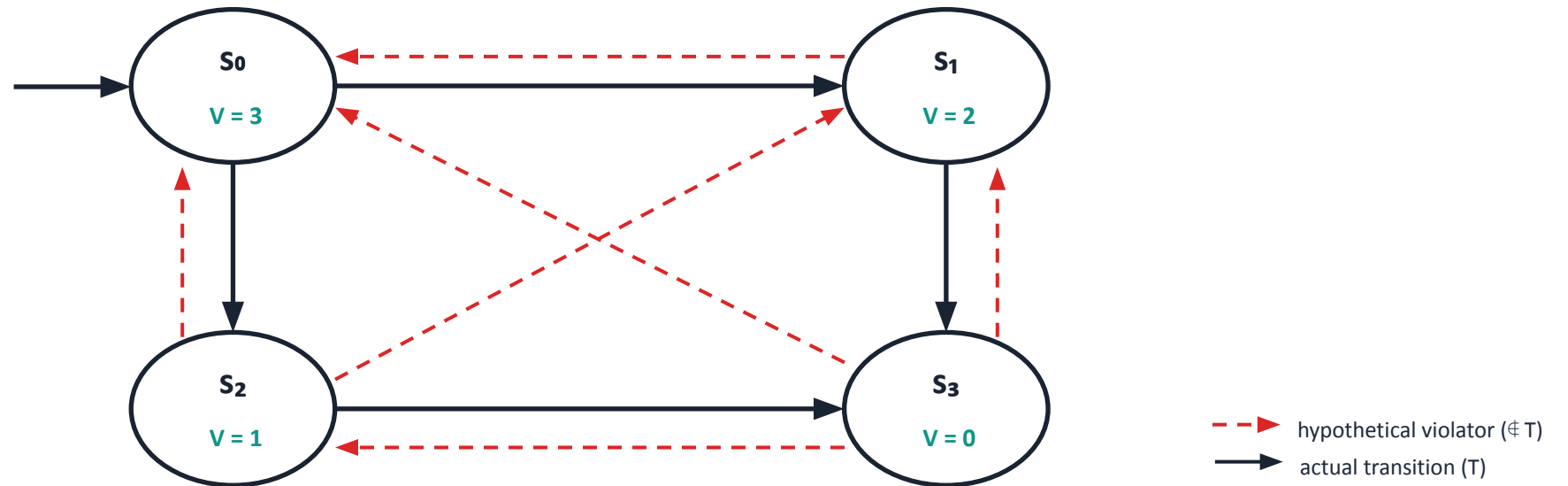
no infinite descent $\rightarrow$ no infinite run

Ranking Functions

For the special case of termination (toy example).

Show that this system terminates.

Prove contraposition!



(i) Initial states have finite V

no run starts at ∞

(ii) V strictly decreases on every transition

no infinite descent $\rightarrow$ no infinite run

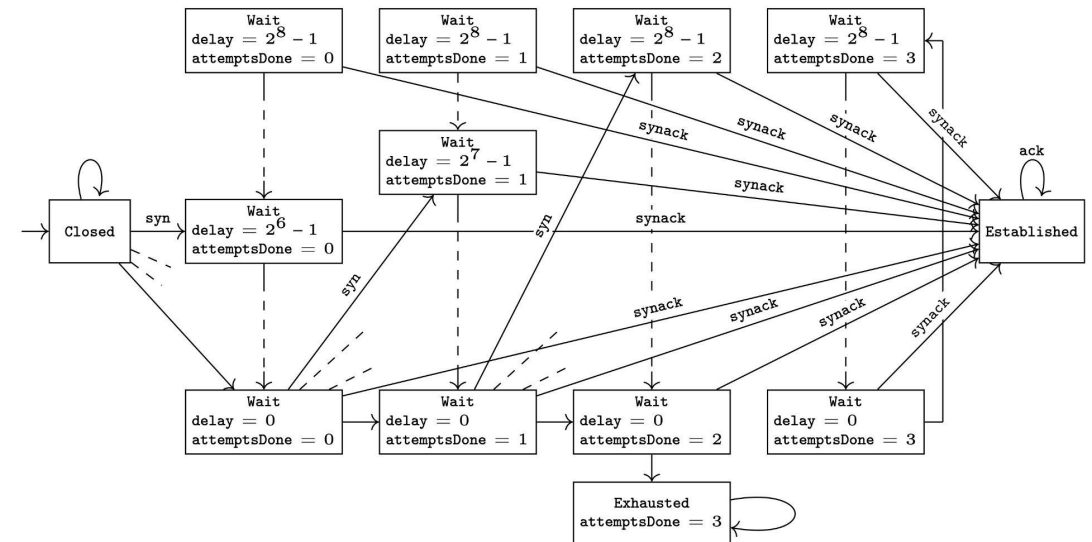
It runs on real protocols (but not AI yet)

3-way TCP-style handshake with exponential backoff. Spec: $\rightarrow F G \text{ Wait}$.

STATE SPACE

2^8

states proved in ~40 minutes



EXTENSION

Supporting infinite state spaces

Towards real AI systems

$$\forall s, s': (s, s') \in T \Rightarrow f(s') < f(s)$$



Difficult in ZK!

Supporting infinite state spaces

Towards real AI systems

$$\forall s, s': (s, s') \in T \Rightarrow f(s') < f(s)$$



Difficult in ZK!

Dual formulation (integer programming):

$$\exists \lambda, \mu \geq 0: \dots \quad (\text{Farka's Lemma})$$



Easy in ZK!



Limited to subclass of
systems.

Initial benchmark

on toy example

Benchmark	S	Explicit prover	Symbolic prover
exb_i1a2	2^5	1 s	4 min
exb_i4a2	2^7	3 min	4 min
exb_i8a2	2^8	40 min	4 min
exb_i4a3	2^8	OOT	5 min
exb_i128a4	2^{15}	—	5 min

Zero Knowledge Model Checking
<https://arxiv.org/abs/2605.00487>



How do we bind the proof to what actually executes?

A ZK proof about a commitment is worthless if you can't prove the deployed system IS that commitment.

zkVMs / zk inference

general, but costly

TEEs + remote attestation

cheap, trust hardware

Model equality in ZK

?

For discussion: Which of these is realistic for frontier AI?

Scaling formal verification/ranking functions to AI

What safety properties can we prove? Do we have efficient certificates for it?

WHAT WE CAN CERTIFY TODAY

Termination, liveness

Reachability of states

Local robustness on $B\epsilon(x_0)$

Lipschitz / monotonicity bounds

Small RNNs with LTL invariants

WHAT WE'D LIKE TO CERTIFY ON AI

Truthfulness

no formal predicate

Refusal on broad categories

CBRN, CSAM — semantic

Non-deception

no spec language exists

Goal-directedness

operational definition unclear

Robustness against jailbreaks

infinite input space

For discussion: Which properties are we interested in? Can a learned certificate substitute?

What does the public spec leak about the secret system?

ZK hides M , but ϕ is public. A cleverly chosen spec might be a side channel.

THE WORRY

A spec is a query into the model.

"**Model never outputs string X on inputs in class Y.**" Knowing this is true narrows the space of possible Ms.

Extreme case: spec fully determines system, nothing remains to hide.

*Example spec:
M computes the fibonacci sequence*

For discussion: Is this a real issue? What's the right formal model?

Three questions for the room

- 01** How do we prove that a **running system** corresponds to a **commitment**?
- 02** Which **AI safety properties** are we interested in and which admit short, checkable certificates?
- 03** Is a **public spec** a real side channel?

Two things people want. In tension.

Assurance

Regulators, auditors, customers, users want a guarantee:

“This avionics software never enters an unsafe state.”

“This protocol never deadlocks.”

“This model satisfies the safety spec.”

Confidentiality

Developers, vendors, providers need to protect:

- intellectual property and source code
- proprietary protocol internals
- model weights and training data
- customer or user data baked into the system

Classical formal verification resolves the tension in one direction: the verifier sees the system.

ZK: three sub-proofs

Pedersen-commit the secrets. Discharge each property with a specialised sigma protocol.

Range proofs

Bulletproofs

The Farkas witness λ , μ lies in $[0, M]$.
The secret matrices A_s , b_s lie in $[-M, M]$.

Matrix multiplication

zkMatrix

Prove products like $A_s^T \lambda$ commit to the claimed value, without revealing A_s or λ .

Equality proofs

DLEQ + Schnorr

Prove $A_s^T \lambda = -G_p^T \mu$. Enforces the Farkas witness really witnesses this obligation.

$$\underbrace{\lambda_s \in [0, M] \wedge \mu_s \in [0, M]}_{\text{range proof}} \wedge \underbrace{-b_s^T \lambda_s - h_p^T \mu_s - 1 \in [0, M]}_{\text{matrix multiplication}} \wedge \underbrace{A_s^T \lambda_s = -G_p^T \mu_s}_{\text{equality proof}}$$

Compose with independent randomness $\rightarrow$ zero-knowledge preserved.

The ZKMC team



Pascal Berrang

Associate Professor, Birmingham ·
Co-founder & CTO, Zeroth Research



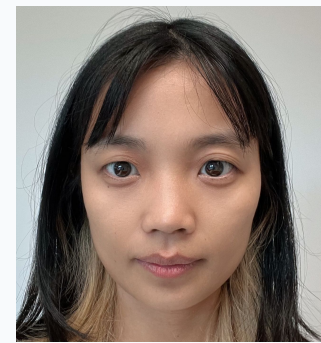
Mirco Giacobbe

Associate Professor, Birmingham
Co-founder & CEO, Zeroth Research



Jacob Swales

PhD student, Birmingham



Xiao Yang

Postdoctoral researcher, Birmingham